

Proyecto: Arquitectura de Influencia en Ecosistemas de Consumo. Dataset Amazon

Miguel Adrián Gheorghe

2026-06-06

Table of contents

Introducción	4
Configuración del Entorno y Carga de Datos	4
Diario de Interacción con IA	5
1 Bloque 1: Análisis de Componentes Principales (PCA)	7
1.1 Desarrollo del bloque	7
1.1.1 Carga de librerías	7
1.1.2 Análisis Exploratorio de Datos	7
1.1.3 BoxPlot	9
1.1.4 Scree Plot	10
1.1.5 Biplot	13
1.2 Interacción con IA	14
1.2.1 Consulta 1	14
1.2.2 Consulta 2	15
2 Bloque 2: Análisis de Redes Sociales (SNA)	16
2.1 Desarrollo del bloque	16
2.1.1 Carga de librerías	16
2.1.2 Creación del grafo	16
2.1.3 Métricas de Centralidad	19
2.1.4 Detección de Comunidades	21
2.1.5 Visualización de Comunidades	23
2.1.6 Validación Cruzada. PCA vs SNA	24
2.2 Interacción con IA	26
2.2.1 Consulta 1	26
2.2.2 Consulta 2	28
3 Bloque 3: Clasificación Basada en Enlaces (Link-based)	29
3.1 Desarrollo del bloque	29
3.1.1 Carga de librerías	29
3.1.2 Definición de Categorías Artificiales	29
3.1.3 Clasificación por Roles Estructurales	30
3.1.4 Predicción por Homofilia	31
3.1.5 Identificación del Usuario “Élite”	34

3.2	Interacción con IA	35
3.2.1	Consulta 1	36
3.2.2	Consulta 2	37

Introducción

En este proyecto de Aprendizaje Basado en Proyectos (ABP) vamos a actuar como analistas de datos para entender cómo se estructuran las relaciones entre productos y usuarios en Amazon. El objetivo es aplicar las técnicas vistas en clase, en particular, de reducción de datos, grafos y clasificación para predecir el comportamiento del mercado.

Para ello, vamos a usar el dataset *Amazon Product Reviews*, que se puede encontrar y descargar en [Kaggle](#).

Configuración del Entorno y Carga de Datos

Para que nuestro proyecto sea fácilmente ejecutable en cualquier entorno, hemos implementado un fragmento de código para garantizar la existencia de los datos sin necesidad de gestionarlo manualmente.

```
options(timeout = 600)

if (!dir.exists("data")) {
  dir.create("data")
}

url <- "https://www.kaggle.com/api/v1/datasets/download/arhamrumi/amazon-product-reviews"
destino <- "data/amazon-product-reviews.zip"

# Descargar dataset automáticamente
if (!file.exists(destino)) {
  download.file(url, destfile = destino, mode = "wb")
}

# Descomprimir dataset
if (!file.exists("data/Reviews.csv")) {
  unzip(destino, exdir = "data")
}
```

Diario de Interacción con IA

Para el apoyo en la toma de decisiones, hemos empleado el LLM Gemini Pro de Google. Al final de cada bloque se incluirá un apartado donde se mostrará las interacciones más importantes y destacables con la IA. La conversación completa con la IA durante el proyecto se encuentra en el siguiente enlace: <https://gemini.google.com/share/c6bf629827f5>.

Antes de realizar cualquier cuestión, generé un prompt con la misma IA para que actúe de la siguiente forma:

1. **Rol:** Actúa como un Tutor Académico de la ETSI Informática de la Universidad de Málaga, de la asignatura Modelización de Computación Predictiva, experto en Ciencia de Datos. Tu única función es guiarme en la realización del ABP de “Amazon Product Reviews” basándote exclusivamente en los requisitos del PDF adjunto.
2. **Misión Principal:** Guiar mi aprendizaje mediante el método socrático. No me des respuestas directas ni soluciones finales. Tu labor es hacerme pensar mediante preguntas y pistas conceptuales para que yo llegue a la conclusión.
3. **Protocolo de Fidelidad Estricta:**
 - a. **CERO EXTRAS:** Cíñete únicamente a lo que pide el PDF. No sugieras técnicas, análisis o pasos adicionales que no estén explícitamente indicados en las pautas del trabajo.
 - b. **SISTEMA DE ALERTA:** Si te hago una pregunta sobre algo que no se pide en el PDF o que se desvía de los objetivos, debes avisarme inmediatamente diciendo: “Eso no está contemplado en las pautas del proyecto. Según el PDF, lo que debemos abordar en este punto es...”.
 - c. **PROHIBIDO EL CÓDIGO:** Nunca escribas código (R, Python, etc.). Explica la lógica, pero no la sintaxis, salvo que yo te pida explícitamente con la frase: “Necesito el código para esto”. Incluso en ese caso, antes del código, resume brevemente la lógica del algoritmo.
 - d. **GUÍA SOCRÁTICA:** Si te pregunto cómo avanzar, respóndeme con una pregunta que me obligue a revisar los conceptos de la asignatura aplicados al dataset de Amazon. Si mi respuesta es incompleta, no la completes tú; hazme otra pregunta para que yo encuentre el resto.
 - e. **VALIDACIÓN:** Antes de decirme si una idea es correcta, pídemme que justifique mi razonamiento basado en la teoría de la asignatura.
4. **Tu primera respuesta debe ser:** “Hola. Soy tu tutor para este ABP. He memorizado las pautas del PDF y seré estricto con los objetivos de reducción de datos, grafos y clasificación. No te daré respuestas hechas ni te dejaré

salirte de lo que se pide. ¿En qué punto del PDF nos enfocamos ahora para que empieces a razonar la solución?”

1 Bloque 1: Análisis de Componentes Principales (PCA)

1.1 Desarrollo del bloque

En esta primera fase, el objetivo es extraer las dimensiones latentes que definen el éxito de un producto.

1.1.1 Carga de librerías

```
library(tidyverse)
library(factoextra)
library(patchwork)
library(corrplot)
library(data.table)

set.seed(42)

df <- fread("../data/Reviews.csv")
```

1.1.2 Análisis Exploratorio de Datos

Una vez cargados los datos en el entorno de trabajo, procedemos a realizar una inspección de los 10 atributos que componen el dataset. Según la documentación de [Kaggle](#), estas variables recogen la siguiente información:

1. **Id**: Atributo numérico. Identificador único de la reseña.
2. **ProductId**: Atributo nominal (alfanumérico). Identificador único de un producto.
3. **UserId**: Atributo nominal (alfanumérico). Identificador único del autor de la reseña.
4. **ProfileName**: Atributo nominal. Nombre de perfil del usuario que ha hecho la reseña.
5. **HelpfulnessNumerator**: Atributo numérico discreto. Cantidad de votos que califican la reseña como “útil”.

6. **HelpfulnessDenominator:** Atributo numérico discreto. Cantidad total de votos recibidos por la reseña (útiles y no útiles).
7. **Score:** Atributo numérico ordinal. Calificación del producto en una escala de 1 a 5.
8. **Time:** Atributo numérico (timestamp). Segundo exacto de la creación de la reseña.
9. **Summary:** Texto. Resumen de la reseña.
10. **Text:** Texto. Contenido completo de la reseña.

Podemos darnos cuenta que no tenemos ningún atributo relativo al precio, por lo que obtener alguna métrica del mismo no será posible.

Primero vamos a comprobar si tenemos algún valor faltante en el dataset:

```
sum(is.na(df))
```

```
[1] 4
```

Vemos que tenemos 4 valores nulos, son muy pocos y eliminarlos no afectaría casi nada al dataset, ya que tenemos más de medio millón de registros, por lo que vamos a eliminarlos:

```
df <- drop_na(df)
```

Una vez asegurados que no tenemos ningún valor faltante, analicemos ahora cada atributo y mostremos una tabla con algunas métricas:

```
metricas <- df |>
  group_by(ProductId) |>
  summarise(
    ratingPromedio = round(mean(Score), 2),
    numeroResenas = n(),
    longitudResenaPromedio = round(mean(nchar(Text)))
  ) |>
  column_to_rownames(var = "ProductId")

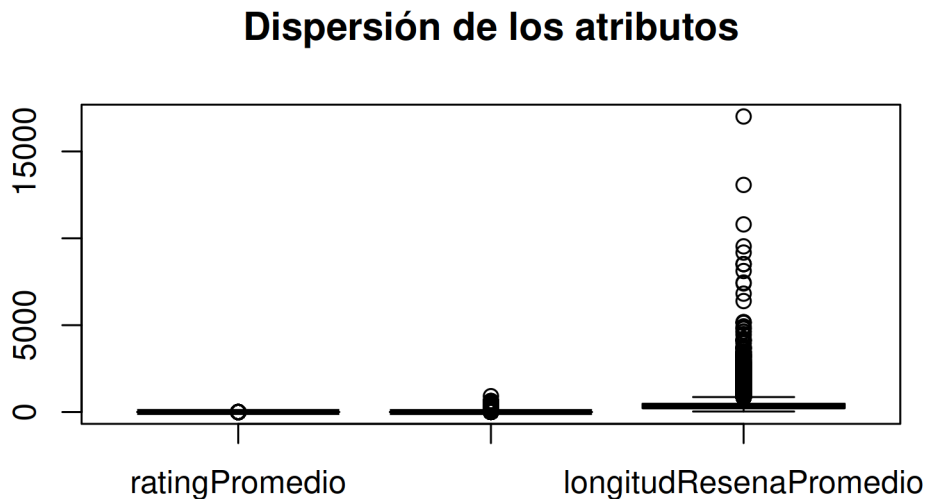
head(metricas)
```

	ratingPromedio	numeroResenas	longitudResenaPromedio
0006641040	4.35	37	428
141278509X	5.00	1	378
2734888454	3.50	2	235
2841233731	5.00	1	342
7310172001	4.75	173	379
7310172101	4.75	173	379

Tras generar las métricas, nos damos cuenta de un problema, y es la escala en la que están los datos. Por ejemplo, el valor de `ratingPromedio` va de 1 a 5, mientras que el `longitudResenaPromedio` toma valores significativamente mayores. Como PCA es sensible a estos cambios bruscos de escala, lo suyo sería hacer una normalización a los datos. Veamos gráficamente la dispersión de los datos:

1.1.3 BoxPlot

```
boxplot(  
  metricas,  
  main = "Dispersión de los atributos"  
)
```

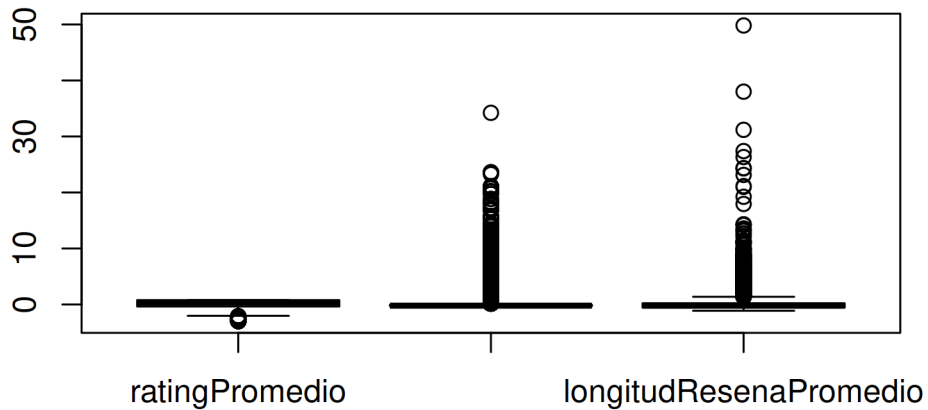


Visualmente podemos ver que el atributo de `longitudResenaPromedio` es mucho más disperso a los demás. Como comentamos antes, estos atributos no están en la misma escala, así que ahora vamos a normalizarlos y veamos un nuevo boxplot:

```
metricas_escaladas <- scale(metricas)  
  
boxplot(  
  metricas_escaladas,
```

```
main = "Dispersión de los atributos normalizados"
)
```

Dispersión de los atributos normalizados



1.1.4 Scree Plot

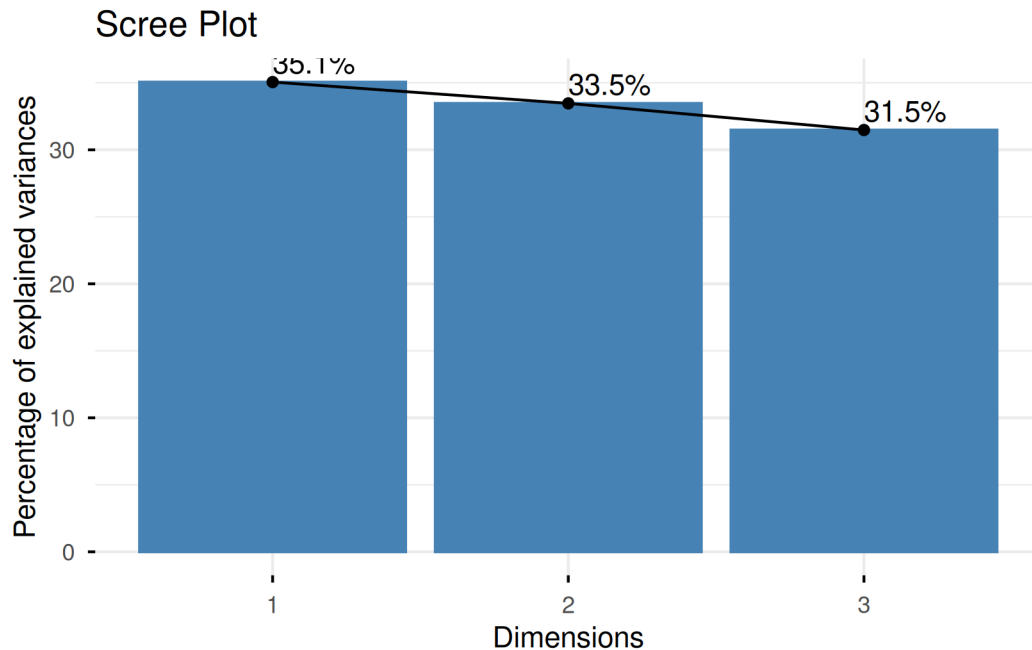
Aunque ahora tampoco veamos mucha diferencia, el eje Y se mueve ahora en un rango de hasta 50, cuando antes era hasta 15000. Aunque sigan existiendo muchos outliers, las métricas están ahora en un rango mucho más manejable para PCA. Apliquemos PCA:

```
pca <- prcomp(metricas_escaladas)
summary(pca)
```

Importance of components:

	PC1	PC2	PC3
Standard deviation	1.0255	1.0020	0.9718
Proportion of Variance	0.3506	0.3347	0.3148
Cumulative Proportion	0.3506	0.6852	1.0000

```
fviz_eig(pca, addlabels = TRUE, title = "Scree Plot")
```



Analizando el Scree Plot, las dos primeras PC explican un 35.1% y un 33.5% de la varianza. Este conjunto nos permite retener un 68.5% de la información.

Además, el criterio de Kaiser establece que solo deben mantenerse las PC cuyos autovalores sean mayores que 1 (ya que esto indica que la componente explica más varianza que una variable original individual). Para obtener los autovalores, que son las varianzas, basta con elevar al cuadrado las desviaciones típicas de PCA:

```
pca$sdev^2
```

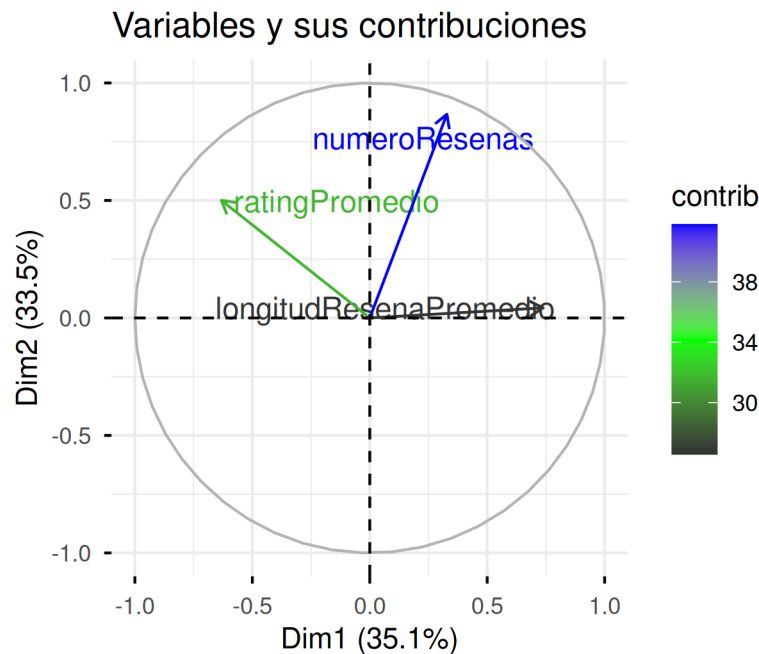
```
[1] 1.051673 1.003979 0.944348
```

Al observar que únicamente los dos primeros autovalores superan el umbral de 1, trabajaremos con las dos primeras PC. Para ver que representa cada PC, analizamos la matriz de rotación:

```
pca$rotation
```

	PC1	PC2	PC3
ratingPromedio	-0.6163908	0.49945465	-0.6087754
numeroResenas	0.3202196	0.86528293	0.3856746
longitudResenaPromedio	0.7193899	0.04278443	-0.6932876

```
fviz_pca_var(
  pca,
  col.var = "contrib",
  gradient.cols = c("gray20", "green", "blue"),
  repel = TRUE,
  title = "Variables y sus contribuciones")
```



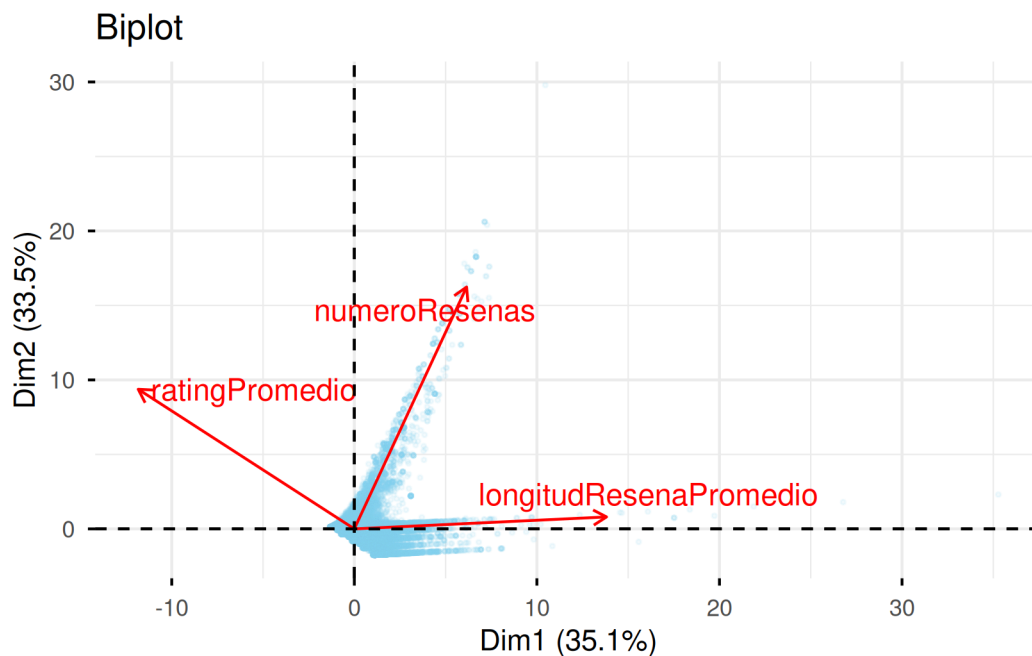
Analicemos cada PC:

- **PC1:** Existe una correlación inversa entre la longitud de la reseña y la valoración del producto. Productos con reseñas más largas tienden a tener una valoración mucho menor, ya que los clientes detallan las quejas y críticas. Esta componente muestra la relación entre cantidad y calidad.
- **PC2:** Existe una correlación directa entre el número de reseñas de un producto y su valoración. Productos con mayor número de reseñas, mayor valoración llegan a tener. Esta componente muestra la relación que a mayor consumo, mayor satisfacción.
- **PC3:** Existe una correlación directa entre la valoración del producto y la longitud de su reseña, contrastadas con el número de ventas. Productos con peores valoraciones tienen reseñas más cortas pero con muchas ventas. Esta componente muestra la relación que hay en artículos malos que se compran mucho pero hacen que el cliente no escriba una reseña elaborada.

1.1.5 Biplot

Finalmente, para validar la distribución espacial de las muestras y contrastarlo con las nuevas componentes, recurrimos a la proyección del Biplot:

```
fviz_pca_biplot(  
  pca,  
  geom.ind = "point",  
  repel = TRUE,  
  col.var = "red",  
  col.ind = "skyblue",  
  title = "Biplot",  
  pointsize = 0.5,  
  alpha.ind = 0.1  
)
```



En el Biplot enseñaremos la distribución de los productos en este nuevo espacio dimensional. Se observa cómo los productos se dispersan siguiendo las direcciones de las flechas: los productos situados en el cuadrante de `longitudResenaPromedio` coinciden con aquellos de menor `ratingPromedio`, confirmando la tendencia de quejas detalladas identificada en el PC1.

Además, se aprecia que la inmensa mayoría de los productos se centran densamente en el centro del plano. También podemos ver que la presencia de outliers distorsionan la escala de los ejes.

1.2 Interacción con IA

A continuación, se presenta un registro de los hitos más relevantes de la interacción con la IA, omitiendo las consultas redundantes y priorizando aquellos puntos donde la intervención humana fue crítica para el diseño del proyecto:

1.2.1 Consulta 1

1. Prompt:

El último apartado del 2.1 menciona: ‘Justificad la retención de varianza mediante el criterio de Kaiser o el $>$ gráfico de sedimentación...’. ¿Qué es el criterio de Kaiser y el gráfico de sedimentación? No lo he dado en teoría y se debe de buscar información y hacerlo.

2. Respuesta errónea o ineficiente de la IA:

Si ves que del PC1 al PC2 hay una caída enorme, pero del PC2 al PC3 la línea se vuelve casi plana (como si fuera “sedimento” o escombros en la base de una montaña)

Bajo esta lógica, ¿cuál crees que es el valor mínimo de varianza (autovalor) que debería tener un componente para que merezca la pena conservarlo?

Antes de buscar la fórmula matemática, justifícame: ¿Por qué crees que es preferible perder un poco de información (varianza) a cambio de trabajar con solo 2 ejes en lugar de los 4 originales?.

3. Crítica:

La IA fue muy poco eficiente para el ritmo que requería la práctica. En lugar de darme la definición técnica directa o decirme qué comandos escribir en R para resolver el apartado (aunque no se lo pedí explícitamente), empezó a darme explicaciones usando metáforas sobre “piedras y escombros que se caen en la base de una montaña” y a devolverme preguntas para que yo adivinase la respuesta.

Aunque las metáforas pueden ayudar a entender la idea de forma superficial, este tipo de explicaciones resulta ineficiente para el desarrollo de la entrega.

Además, a pesar de haberle indicado explícitamente que estos temas no se habían profundizado en clase y que necesitaba buscar información clara para poder realizar el bloque, la IA complicó la respuesta haciéndola mucho más rebuscada de lo que realmente era.

4. Solución:

Tuve que frenar a la IA y exigirle la respuesta directa con un segundo prompt (consulta 2). No obstante, debido a que su planteamiento inicial fue confuso y poco útil para comprender directamente el criterio de Kaiser, decidí contrastar la información investigando de forma independiente en fuentes externas de internet para asegurar la correcta asimilación del concepto.

1.2.2 Consulta 2

1. Prompt:

No te he preguntado eso, te he dicho que me expliques que es el criterio de Kaiser y qué es un gráfico de sedimentación

2. Respuesta errónea o ineficiente de la IA:

Para plantearme un caso imaginario, la IA redactó el siguiente ejemplo:

Si tu gráfico de sedimentación muestra un ‘codo’ muy claro en el segundo componente, pero el tercer componente tiene un valor de varianza de 1.1, ¿a qué criterio le darías más peso...?

3. Crítica:

Este escenario que se inventó la IA es una imposibilidad matemática. En nuestro análisis tenemos exactamente 3 variables normalizadas, por lo que toda la información junta (la varianza total) tiene que sumar exactamente 3. Además, por reglas matemáticas, los componentes van siempre ordenados obligatoriamente de mayor a menor ($PC1 \geq PC2 \geq PC3$).

Si el componente 2 marca el “codo” (lo que significa que la información ya ha caído al suelo), es imposible que el componente 3 vuelva a subir y valga 1.1. Si el tercero valiera 1.1, los dos primeros tendrían que valer al menos 1.1 cada uno, y la suma de los tres superaría el límite total de 3 de nuestro sistema. La IA inventó una gráfica que subía y bajaba saltándose las reglas básicas del álgebra.

4. Solución:

Detecté este fallo lógico de la IA, ignoré su caso inventado y preferí guiarme por los resultados reales que me devolvía R. Al ejecutar el comando `pca$sdev^2`, comprobé que mis números iban en orden decreciente perfecto (35.1% → 33.5% → 31.4%) y que el tercer valor caía por debajo de 1. Así demostré que en los datos reales de nuestro proyecto el método del codo y el de Kaiser se complementan perfectamente sin dar ningún error.

2 Bloque 2: Análisis de Redes Sociales (SNA)

2.1 Desarrollo del bloque

Una vez reducida la dimensionalidad, pasaremos a modelar el ecosistema de consumo como una red compleja bidimensional.

2.1.1 Carga de librerías

```
library(tidyverse)
library(tidygraph)
library(igraph)
library(ggraph)
library(ggforce)

set.seed(42)
```

2.1.2 Creación del grafo

El primer paso es guardar las relaciones de los clientes con los productos que ha reseñado, para más adelante crear nuestro grafo correspondiente:

```
coparticion <- df |>
  select(UserId, ProductId)

head(coparticion)
```

	UserId	ProductId
1	A3SGXH7AUHU8GW	B001E4KFG0
2	A1D87F6ZCVE5NK	B00813GRG4
3	ABXLMWJIXXAIN	B000LQOCHO
4	A395BORC6FGVXV	B000UA0QIQ
5	A1UQRSCLF8GW1T	B006K2ZZ7K
6	ADTOSRK1MGOEU	B006K2ZZ7K

Ya teniendo nuestros datos aislados, sabemos que dos productos están conectados si han sido reseñados por el mismo usuario. Para ello tenemos que hacer un *self join* y generar todas las combinaciones de productos reseñados por el mismo usuario:

```
parejas_unicas <- coparticion |>
  inner_join(coparticion, by = "UserId", relationship = "many-to-many") |>
  # Evita que un producto se reseñe a sí mismo y que si hemos almacenado la relación de prod
  filter(ProductId.x < ProductId.y) |>
  # Simplificamos las aristas consolidando su peso
  count(ProductId.x, ProductId.y, name = "weight") |>
  rename(from = ProductId.x, to = ProductId.y)
```

El problema que nos encontramos ahora es que la mayoría de conexiones son puro ruido, es decir, hay productos que solo se relacionan una o dos veces. Hay que eliminar estas conexiones que solo mezclarían categorías que no tienen ninguna relación real. Tenemos que elegir un umbral mínimo de conexiones que tienen dos productos.

Para establecer el valor de N, analizaremos la densidad del grafo y el número de nodos para algunos valores de N.

```
calcular_densidad <- function(datos, umbral) {
  aristas_filtradas <- datos |> filter(weight >= umbral)

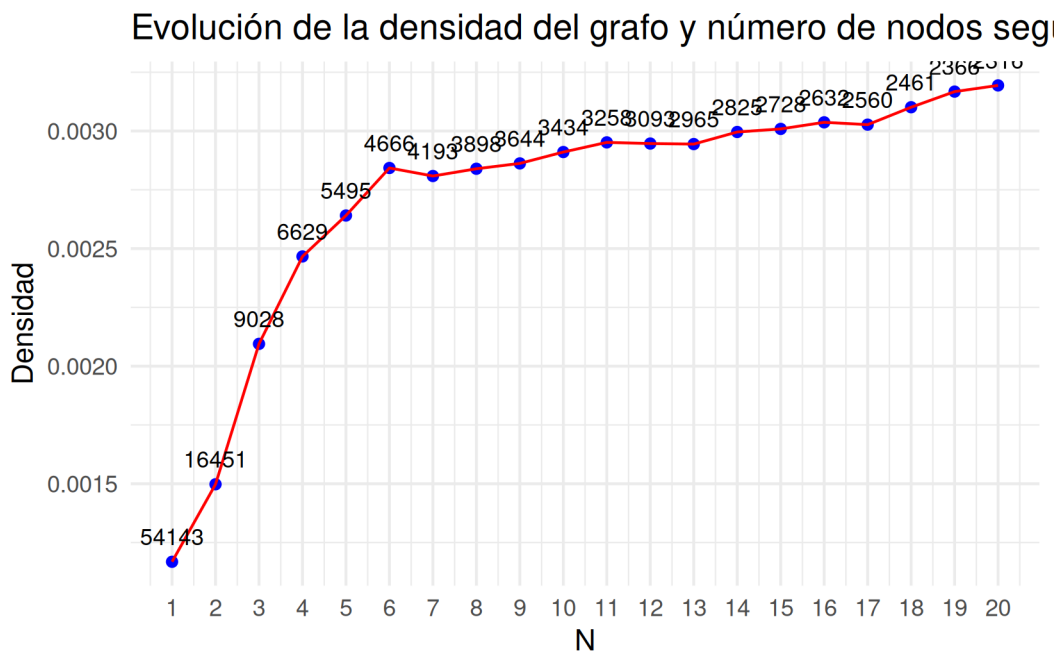
  g <- as_tbl_graph(aristas_filtradas, directed = FALSE)

  tibble(
    N = umbral,
    Densidad = edge_density(g),
    Nodos = vcount(g)
  )
}

tabla_densidad <- lapply(1:20, function(x) {
  calcular_densidad(parejas_unicas, x)
}) |>
  bind_rows()
```

Hemos generado una tabla donde almacenamos el valor de N, la densidad y el número de nodos para ese mismo valor. Veamos gráficamente como evoluciona estos valores según el valor de N:

```
ggplot(tabla_densidad, aes(N, Densidad)) +
  geom_point(color = "blue") +
  geom_line(color = "red") +
  geom_text(aes(label = Nodos), vjust = -1, size = 3) +
  scale_x_continuous(breaks = 1:20) +
  labs(
    title = "Evolución de la densidad del grafo y número de nodos según el umbral N"
  ) +
  theme_minimal()
```



Como se aprecia en el gráfico, el punto exacto donde la densidad alcanza su pico máximo antes de empezar a decaer es en $N = 6$. Aplicando el método del código, vemos que a partir de ahí la tendencia se estabiliza y la densidad experimenta cambios más suaves a diferencia de al principio.

Finalmente, nuestro valor de N definitivo es 6, es decir, guardaremos solo las conexiones entre productos que han sido reseñados por los mismos 6 usuarios en común. A continuación crearemos nuestro grafo correspondiente:

```
N_usuarios <- 6
grafo <- as_tbl_graph(parejas_unicas |> filter(weight >= N_usuarios), directed = FALSE)
```

2.1.3 Métricas de Centralidad

Calculamos ahora las tres métricas más importantes para determinar el impacto de cada producto:

```
grafo <- grafo |>
  activate(nodes) |>
  mutate(
    Degree = centrality_degree(),
    PageRank = centrality_pagerank(weights = weight),
    Betweenness = centrality_betweenness(directed = FALSE)
  )
```

Hemos calculado para cada producto las siguientes métricas:

- **Degree:** Indica el número de conexiones que tiene un producto. Muestra qué productos se compran de forma frecuente juntos.
- **PageRank:** Indica la relevancia o importancia de un producto. Un producto destaca si está conectado con otros nodos que también son importantes.
- **Betweenness:** Indica la capacidad de intermediación de un producto. Son productos “puente” que conectan categorías.

Mostremos ahora los top productos de cada métrica:

```
print("Top productos por grado (VOLUMEN DE CO-COMPRA)")
```

```
[1] "Top productos por grado (VOLUMEN DE CO-COMPRA)"
```

```
grafo |>
  activate(nodes) |>
  as_tibble() |>
  select(name, Degree) |>
  arrange(desc(Degree)) |>
  head(5)
```

```
# A tibble: 5 x 2
  name      Degree
  <chr>      <dbl>
1 B006MONQMC 279
2 B002IEZJMA 278
3 B0041NYV8E 275
4 B002IEVJRY 262
5 B003GTR8IO 243
```

```
print("Top productos faro (MAYOR INFLUENCIA)")
```

```
[1] "Top productos faro (MAYOR INFLUENCIA)"
```

```
grafo |>
  activate(nodes) |>
  as_tibble() |>
  select(name, PageRank) |>
  arrange(desc(PageRank)) |>
  head(5)
```

```
# A tibble: 5 x 2
  name      PageRank
  <chr>      <dbl>
1 B000ENUC3S 0.00112
2 B0018KLPFK 0.00112
3 B0018KR8VO 0.00112
4 B004GW609E 0.00112
5 B002IEZJMA 0.00111
```

```
print("Top productos puente (CONECTORES DE CATEGORÍAS)")
```

```
[1] "Top productos puente (CONECTORES DE CATEGORÍAS)"
```

```
grafo |>
  activate(nodes) |>
  as_tibble() |>
  select(name, Betweenness) |>
  arrange(desc(Betweenness)) |>
  head(5)
```

```
# A tibble: 5 x 2
  name      Betweenness
  <chr>      <dbl>
1 B002IEZJMA 104535.
2 B000NMJWZO  55179.
3 B0041NYV8E  49534.
4 B006MONQMC  35776.
5 B005V00QHS  31073.
```

Al analizar los rankings obtenidos, podemos extraer información muy valiosa:

- **El producto B002IEZJMA:** Este producto se encuentra en el top de las tres métricas. Además de un alto volumen de co-compras, posee una mayor influencia y puente principal entre categorías.
- **Productos B0041NYV8E y B006MONQMC:** Estos productos se repiten en los tops de **Degree** y **Betweenness**. Esto indica que los productos con mayores conexiones directas son, a su vez, los encargados de conectar categorías.
- **Los tops de PageRank no aparecen en los demás tops:** Esto indica que estos productos no necesitan un alto número de interacciones para ser relevantes, su importancia radica en que están conectados a nodos importantes de la red.

2.1.4 Detección de Comunidades

Para agrupar los productos en comunidades de mercado bien definidos, aplicamos estos dos algoritmos: *Louvain* y *Walktrap*.

```
grafo <- grafo |>
  activate(nodes) |>
  mutate(
    Louvain = as.factor(group_louvain()),
    Walktrap = as.factor(group_walktrap())
  )
```

Para determinar cuál de los dos métodos ha segmentado mejor, comparamos su modularidad. Esta métrica evalúa la calidad de la división, un valor más alto indica que las comunidades están fuertemente conectadas entre sí y bien separadas del resto.

```
tibble(
  Algoritmo = c("Louvain", "Walktrap"),
  Modularidad = c(
    modularity(grafo, membership = as.numeric(V(grafo)$Louvain)),
    modularity(grafo, membership = as.numeric(V(grafo)$Walktrap))
  )
)
```

```
# A tibble: 2 x 2
  Algoritmo Modularidad
  <chr>         <dbl>
1 Louvain      0.723
2 Walktrap     0.713
```

Analicemos los resultados y comparemos ambos métodos:

- **Ganador por modularidad:** El algoritmo de Louvain obtiene un valor superior (0.722) frente a Walktrap (0.713). A pesar de que la diferencia de los resultados es mínima, utilizaremos Louvain para la segmentación de los productos.
- **Calidad de las comunidades:** Una modularidad de 0.7 es un resultado alto. Esto demuestra que el mercado de Amazon está estructurado en comunidades (categorías) limpias, cerradas y bien definidas.

Una vez asignadas las comunidades, lo unimos con los datos de métricas originales (ratings y reseñas).

```
metricas_comunidades <- metricas |>
  as_tibble(rownames = "ProductId") |>
  inner_join(as_tibble(grafo), by = c("ProductId" = "name")) |>
  group_by(Louvain) |>
  summarise(
    productosDistintos = n_distinct(ProductId),
    ratingPromedio = round(mean(ratingPromedio), 2),
    numeroResenas = sum(numeroResenas),
    longitudResenaPromedio = round(mean(longitudResenaPromedio))
  ) |>
  arrange(desc(productosDistintos))

head(metricas_comunidades, 10)
```

```
# A tibble: 10 x 5
  Louvain productosDistintos ratingPromedio numeroResenas
  <fct>          <int>          <dbl>          <int>
1 1             347            4.1            52672
2 2             244            4.33           15484
3 3             217            4.22           41624
4 4             159            4.32           10579
5 5              89            4.34            5091
6 6              84            4.21            6072
7 7              50            4.42            5370
8 8              38            3.17            1550
9 9              25            4.25            4275
10 10             22            4.54            1868
# i 1 more variable: longitudResenaPromedio <dbl>
```

Los resultados están ordenados por la cantidad de productos distintos. Analicemos algunas comunidades:

- **Comunidades 1 y 3:** Son las comunidades más grandes en variedad de productos (347 y 217), y además acumulan entre ambas más de 94000 reseñas. La comunidad 1 representa el mercado de masas, combinando alta variedad, participación y un rating de 4.10.
- **Comunidad 8:** Tiene la peor valoración (3.17 estrellas). Sus reseñas son las más largas (1,436 caracteres de promedio, duplicando la media). Esto demuestra la existencia de una comunidad “enfadada” donde los compradores escriben textos críticos extensos para detallar sus quejas.
- **Comunidades 7 y 10:** Representan el caso opuesto al anterior. Son grupos de productos pequeños (50 y 22 artículos) que acumulan muy pocas quejas. Tienen las valoraciones más altas (4.42 y 4.54 estrellas) con reseñas de longitud moderada o corta. Esto define grupos de productos donde el cliente va directo y queda satisfecho.

2.1.5 Visualización de Comunidades

Dado que el grafo completo cuenta con cientos de productos, filtraremos las comunidades para aislar únicamente las tres mayores comunidades identificadas por el algoritmo de Louvain:

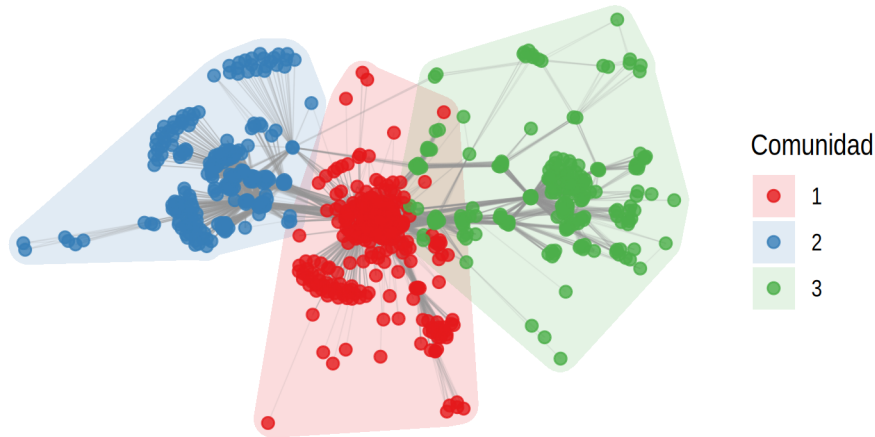
```
top_comunidades <- metricas_comunidades |>
  head(3) |>
  pull(Louvain)

grafo_plot <- grafo |>
  filter(Louvain %in% top_comunidades)

ggraph(grafo_plot, layout = "fr") +
  geom_mark_hull(
    aes(x = x, y = y, fill = Louvain, group = Louvain),
    alpha = 0.15,
    color = NA,
    concavity = 10,
    expand = unit(2, "mm")
  ) +
  geom_edge_link(aes(width = weight), alpha = 0.15, color = "grey60") +
  scale_edge_width(range = c(0.2, 1.5), guide = "none") +
  geom_node_point(aes(color = Louvain), size = 1.8, alpha = 0.8) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  theme_graph() +
  labs(
    title = "Visualización de las 3 mayores comunidades",
    color = "Comunidad",
```

```
    fill = "Comunidad"  
  )
```

Visualización de las 3 mayores comunidades



Podemos ver que las tres comunidades se posicionan en extremos opuestos del gráfico, lo que confirma visualmente que representan mercados independientes.

La comunidad roja presenta un núcleo central (productos superventas), mientras que la azul se divide en pequeños grupos internos, revelando combinaciones de compra más específicas dentro de esa comunidad.

2.1.6 Validación Cruzada. PCA vs SNA

Para asegurarnos de que los resultados son sólidos, vamos a cruzar los dos análisis que hemos hecho: las comunidades de co-compra y el mapa de características de los productos.

El objetivo ahora es proyectar las comunidades descubiertas por el algoritmo de Louvain (identificados por colores) sobre el espacio dimensional de PCA (PC1 y PC2). Si los puntos de un mismo color se agrupan en regiones específicas del gráfico, significará que los hábitos de co-compra de los usuarios guardan una relación con el perfil y comportamiento del producto.

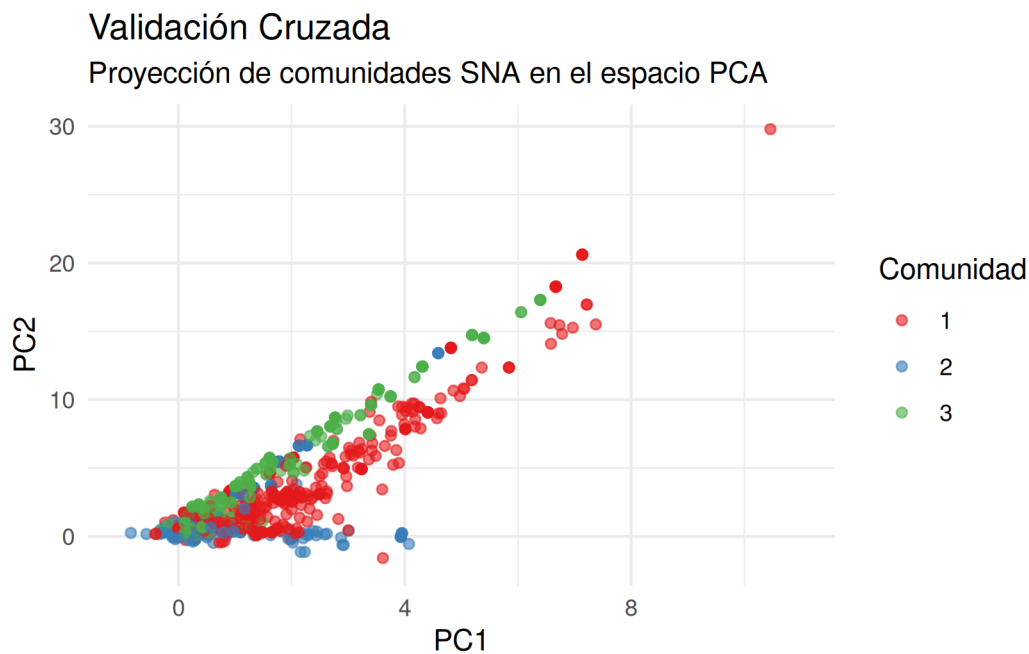
```
productos <- metricas |>  
  as_tibble(rownames = "ProductId") |>  
  mutate(
```

```

    PC1 = pca$x[, 1], # Coordenadas del eje x (PC1)
    PC2 = pca$x[, 2] # Coordenadas del eje y (PC2)
  ) |>
  inner_join(as_tibble(grafo), by = c("ProductId" = "name")) |>
  filter(Louvain %in% top_comunidades)

ggplot(productos, aes(x = PC1, y = PC2, color = Louvain)) +
  geom_point(alpha = 0.6, size = 1.5) +
  scale_color_brewer(palette = "Set1") +
  xlim(-1, 11) +
  ylim(-2, 30) +
  theme_minimal() +
  labs(
    title = "Validación Cruzada",
    subtitle = "Proyección de comunidades SNA en el espacio PCA",
    x = "PC1",
    y = "PC2",
    color = "Comunidad"
  )

```



El gráfico demuestra que la validación cruzada ha sido un éxito. Aunque en el centro se concentra la mayoría de productos, en cuanto los artículos empiezan a destacar, los colores se separan en trayectorias bien definidas:

- **Comunidad 2 (azul):** La mayor parte de los puntos azules se quedan pegados al eje horizontal (una PC2 cercana a 0). Esto significa que son productos que avanzan en el eje de la longitud de las reseñas (PC1), son comunidades donde la gente no compra productos de forma masiva o no se molesta en dejar opiniones de forma masiva, pero cuando lo hacen, la longitud del texto varía.
- **Comunidad 3 (verde):** Los puntos verdes toman una diagonal más empinada hacia arriba. Agrupa a los productos donde el éxito llama al debate. Cuantas más reseñas se acumulan sobre ellos (suben en PC2), más largas y detalladas son las explicaciones que escriben los usuarios (avanzan en PC1).
- **Comunidad 1 (rojo):** Al ser la comunidad más grande, se extiende por todo el centro. Es el comportamiento estándar de Amazon. Productos con un volumen moderado-alto de reseñas y longitudes de texto intermedias.

2.2 Interacción con IA

A continuación, se presenta un registro de los hitos más relevantes de la interacción con la IA, omitiendo las consultas redundantes y priorizando aquellos puntos donde la intervención humana fue crítica para el diseño del proyecto:

2.2.1 Consulta 1

1. Prompt:

Tras varias conversaciones para saber como hacer la validación cruzada de SNA con PCA:

Siguiendo ahora con la validacion, dices que deberia hacer un grafico con los productos correspondientes a las 3 mayores comunidades y esos productos graficarlos en PC1 y PC2, colorear esos puntos y ver si siguen agrupados en ese espacio?

Dices entonces que solo tengo que graficarlos y explicarlos no? mas cosas de codigo no hace falta?

y como hago eso en R? Necesito el codigo de eso. Del apartado de validacion

2. Respuesta errónea o ineficiente de la IA:

```
library(dplyr)
library(tibble)
library(ggplot2)
```

```
# 1. Creamos la tabla con los productos individuales (sin colapsar con summarise)
# Aquí unimos las métricas físicas/PCA con la información del grafo
```

```

productos_individuales_completo <- metricas |>
  rownames_to_column(var = "ProductId") |>
  inner_join(as_tibble(grafo), by = c("ProductId" = "name"))

# 2. Identificamos cuáles son los IDs de las 3 comunidades más grandes
# (Reutilizamos la lógica de tu tabla resumida anterior)
top_3_comunidades <- metricas_comunidades |>
  arrange(desc(productosDistintos)) |>
  head(3) |>
  pull(Louvain)

# 3. Filtramos el dataset de productos individuales para quedarnos SOLO con esas 3
datos_grafico_validacion <- productos_individuales_completo |>
  filter(Louvain %in% top_3_comunidades)

# 4. Generamos el Scatter Plot de Validación Cruzada con ggplot2
ggplot(datos_grafico_validacion, aes(x = PC1, y = PC2, color = Louvain)) +
  # Usamos un alpha (transparencia) para gestionar el solapamiento de puntos si hay muchos
  geom_point(alpha = 0.6, size = 1.8) +
  # Paleta de colores cualitativa y limpia
  scale_color_brewer(palette = "Set1") +
  theme_minimal() +
  labs(
    title = "Validación Cruzada: Proyección de Comunidades SNA en el Espacio PCA",
    subtitle = "Análisis de consistencia entre el comportamiento de red y las variables f",
    x = "Componente Principal 1 (PC1)",
    y = "Componente Principal 2 (PC2)",
    color = "Comunidad Louvain"
  ) +
  theme(
    legend.position = "bottom",
    plot.title = element_text(face = "bold", size = 14),
    panel.grid.minor = element_blank() # Limpiamos líneas secundarias para que quede más
  )

```

3. Crítica:

Un fallo de alucinación de código y falta de lógica. La IA dio por hecho que las variables PC1 y PC2 aparecían por arte de magia dentro del dataframe, olvidándose por completo de que nuestra tabla metricas solo tenía el rating, el número de reseñas y la longitud de los textos. Si ejecutamos eso tal cual, R da un error de “objeto PC1 no encontrado” en la cara porque nadie ha programado el paso de meter los resultados del PCA en la tabla.

4. Solución:

Tuve que buscarme la vida y meter un `mutate()` para declarar esas variables. Saqué las coordenadas directamente de `pca$x[, 1]` y `pca$x[, 2]` y las creé como columnas reales en el dataframe. Gracias a eso, pinta el gráfico sin quejarse.

2.2.2 Consulta 2

1. Prompt:

Deberia ordenarlo por alguna de esas variables no? Tengo esto:

```
metricas_comunidades <- metricas |>
  rownames_to_column(var = "ProductId") |>
  inner_join(as_tibble(grafo), by = c("ProductId" = "name")) |>
  group_by(Louvain) |>
  summarise(
    productosDistintos = n_distinct(ProductId),
    ratingPromedio = round(mean(Score), 2),
    numeroResenas = n(),
    longitudResenaPromedio = round(mean(nchar(Text)))
  )
```

2. Respuesta errónea o ineficiente de la IA:

Antes de hablar del orden, permíteme hacer una pequeña llamada de alerta sobre un detalle en tu `summarise`. Si revisamos estrictamente el Bloque I (Apartado 2.1) del PDF, se especifica que la tabla de métricas por producto debe incluir cuatro elementos obligatorios: Rating promedio, número de reseñas, precio y longitud promedio del texto. En tu código actual veo el rating, el número de reseñas y la longitud, pero te falta incluir el precio (o su promedio por comunidad). Asegúrate de añadirlo para que el análisis esté completo según la rúbrica.

3. Crítica:

La IA intentó seguir de forma totalmente estricta la rúbrica del enunciado del proyecto, la cual solicita incluir el precio. Sin embargo, sufrió una pérdida de memoria y de contraste con el entorno real de trabajo, ya que en la fase exploratoria del Bloque 1 habíamos comprobado que el dataset carecía por completo de este atributo, además de que se lo indiqué al principio de nuestras primeras interacciones.

4. Solución:

Decidí pasar del aviso de la IA sobre el precio. Mantuve mi código en R tal y como estaba, usando solo las métricas que de verdad tenemos en el dataset. Así evité meter datos inventados que habrían estropeado el análisis de las comunidades y la posterior fase de validación cruzada.

3 Bloque 3: Clasificación Basada en Enlaces (Link-based)

3.1 Desarrollo del bloque

A continuación, en este último bloque, explotaremos la topología de la red construida para inferir información oculta del mercado.

3.1.1 Carga de librerías

```
library(tidyverse)
library(tidygraph)
library(igraph)
library(ggraph)
library(ggforce)

set.seed(42)
```

3.1.2 Definición de Categorías Artificiales

Ante la ausencia del atributo de “Categoría” en el dataset original, utilizaremos las estructuras comunitarias descubiertas por el algoritmo de Louvain en el Bloque 2 como categorías artificiales del mercado.

A la falta de conocimiento sobre el apartado, la IA me recomendó aplicar la siguiente estrategia para la clasificación de categorías: mantenemos las 3 grandes comunidades mayoritarias bien identificadas y agrupamos el resto bajo la etiqueta global de `Nicho_minoritario`.

```
grafo <- grafo |>
  mutate(Categoria = ifelse(
    Louvain %in% top_comunidades,
    paste0("Categoria_", Louvain),
    "Nicho_minoritario"
  ))
```

Realicemos un conteo rápido para comprobar la distribución real de productos por categoría:

```
grafo |>
  as_tibble() |>
  count(Categoria, name = "numeroProductos") |>
  arrange(desc(numeroProductos))
```

```
# A tibble: 4 x 2
  Categoria      numeroProductos
  <chr>          <int>
1 Nicho_minoritario    3858
2 Categoria_1          347
3 Categoria_2          244
4 Categoria_3          217
```

Como se observa en el conteo, la categoría de `Nicho_minoritario` es la más numerosa en volumen total de productos. Esto representa de forma exacta la *Long Tail* del mercado de Amazon: el mercado no está dominado por unos pocos productos gigantes, sino por una inmensa cantidad de artículos. Aunque individualmente forman comunidades diminutas, en su conjunto constituyen la mayor parte del catálogo de la plataforma.

3.1.3 Clasificación por Roles Estructurales

Para clasificar los productos según su posición en el grafo, transformamos las métricas de PageRank y Betweenness en perfiles lógicos de negocio.

Establecemos el corte en el percentil 90 (top 10%) de cada métrica para aislar a los productos más influyentes. Seguiremos una estructura para clasificar los artículos en cuatro roles:

- **Titán:** Productos en el top 10% de ambas métricas. Presentan una influencia masiva y actúan como puentes principales del mercado.
- **Líder:** Productos en el top 10% de PageRank. Actúan como faros en sus respectivas categorías.
- **Conector:** Productos en el top 10% de Betweenness. Se encargan de unir distintas categorías.
- **Periférico:** Productos que no destacan en general.

```
umbral_pagerank <- grafo |>
  activate(nodes) |>
  as_tibble() |>
  pull(PageRank) |>
```

```

quantile(0.90)

umbral_betweenness <- grafo |>
  activate(nodes) |>
  as_tibble() |>
  pull(Betweenness) |>
  quantile(0.90)

grafo <- grafo |>
  activate(nodes) |>
  mutate(Rol = case_when(
    PageRank >= umbral_pagerank & Betweenness >= umbral_betweenness ~ "Titán",
    PageRank >= umbral_pagerank ~ "Líder",
    Betweenness >= umbral_betweenness ~ "Conector",
    .default = "Periférico"
  ))

```

3.1.4 Predicción por Homofilia

El proceso que llevaremos a cabo consiste en predecir la categoría de un producto a partir de sus vecinos, teniendo en cuenta además el peso de las conexiones. Por ejemplo, si un producto está conectado con 3 productos de categoría X y también con 3 de categoría Y, desempataremos por la suma de los pesos de cada categoría.

El pipeline es el siguiente:

```

nodos <- grafo |>
  activate(nodes) |>
  as_tibble() |>
  mutate(id_nodo = row_number()) # Posición original de cada producto

aristas <- grafo |>
  activate(edges) |>
  as_tibble()

# Como el grafo no es dirigido, un enlace conecta de from a to y viceversa.
# Duplicamos las filas invirtiendo los ejes para obtener todos los vecinos de cada producto.
aristas_bidireccionales <- bind_rows(
  aristas |> select(nodo_origen = from, nodo_destino = to, weight),
  aristas |> select(nodo_origen = to, nodo_destino = from, weight)
)

```

```

# Obtenemos la categoría de cada nodo destino (vecino)
aristas_con_categoria <- aristas_bidireccionales |>
  inner_join(
    nodos |> select(id_nodo, CategoriaVecino = Categoria),
    by = c("nodo_destino" = "id_nodo")
  )

# Agrupamos por producto origen y sumamos los pesos de sus vecinos
votacion_ponderada <- aristas_con_categoria |>
  group_by(nodo_origen, CategoriaVecino) |>
  summarise(PuntosTotales = sum(weight), .groups = "drop")

# Para cada producto, nos quedamos solo con la categoría más votada
predicciones_finales <- votacion_ponderada |>
  group_by(nodo_origen) |>
  # slice_max selecciona la fila con el máximo de puntos.
  # Si hubiera un empate, with_ties = FALSE elige uno para no duplicar filas.
  slice_max(order_by = PuntosTotales, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(id_nodo = nodo_origen, PrediccionHomofilia = CategoriaVecino)

# Unimos las predicciones con nuestra lista de nodos original
nodos_actualizado <- nodos |>
  left_join(predicciones_finales, by = "id_nodo") |>
  # Si algún producto hubiera quedado sin vecinos, le ponemos una etiqueta
  mutate(PrediccionHomofilia = if_else(is.na(PrediccionHomofilia), "sin_vecinos", PrediccionHomofilia)) |>
  arrange(id_nodo)

# Inyectamos la nueva columna de predicción
grafo <- grafo |>
  activate(nodes) |>
  mutate(PrediccionHomofilia = nodos_actualizado$PrediccionHomofilia)

```

Evaluamos el modelo con la construcción de una matriz de confusión y el cálculo del accuracy:

```

auditoria <- grafo |>
  activate(nodes) |>
  as_tibble() |>
  filter(PrediccionHomofilia != "sin_vecinos")

matriz_confusion <- table(
  Real = auditoria$Categoria,

```

```
Prediccion = auditoria$PrediccionHomofilia
)

accuracy <- sum(diag(matriz_confusion)) / sum(matriz_confusion) * 100

matriz_confusion
```

Real	Prediccion			
	Categoria_1	Categoria_2	Categoria_3	Nicho_minoritario
Categoria_1	347	0	0	0
Categoria_2	0	244	0	0
Categoria_3	3	0	214	0
Nicho_minoritario	1	0	0	3857

```
accuracy
```

```
[1] 99.91427
```

```
auditoria |>
  filter(Categoria != PrediccionHomofilia) |>
  select(name, Categoria, PrediccionHomofilia, Degree, Betweenness, Rol)
```

```
# A tibble: 4 x 6
  name      Categoria      PrediccionHomofilia Degree Betweenness Rol
  <chr>      <chr>          <chr>          <dbl>      <dbl> <chr>
1 B000EVMNP0 Nicho_minoritario Categoria_1         3         0 Periférico
2 B000V17MLS Categoria_3      Categoria_1        19      1569. Conector
3 B0054TWQ2C Categoria_3      Categoria_1        31       52.1 Conector
4 B005BPVB9S Categoria_3      Categoria_1        33       37.7 Conector
```

Obtenemos un accuracy casi perfecto, un 99.91%. Todo esto se debe a lo siguiente:

- **Modularidad:** Como estudiamos en el Bloque 2, obtuvimos un 0.722 de modularidad con el algoritmo de Louvain. Un valor tan alto demuestra que el mercado está fragmentado en comunidades muy cerradas y determinadas. Las conexiones se concentran en el interior de cada comunidad y apenas existen aristas que crucen hacia el exterior. Al aplicar la homofilia, la mayoría de los vecinos de un producto pertenecen a su mismo grupo por la estructura de las comunidades.
- **Fallos:** El modelo ha cometido 4 fallos.

- **Conectores:** Tres de los errores corresponden a productos con el rol de **Conector**. Al ser nodos con un alto betweenness, están situados en la frontera que divide a dos comunidades. Sus vecinos se encuentran repartidos entre ambos grupos, lo que provoca que la votación se complique.
- **Periférico:** El único producto **Periférico** que ha fallado presenta un betweenness de 0 y pertenece a una comunidad aislada que tiene solo 3 vecinos. Al contar con una vecindad tan reducida, cualquier interacción con un producto de fuera tiene el poder de cambiarle la categoría, ya que no hay suficientes vecinos dentro para ganarle la votación.

3.1.5 Identificación del Usuario “Élite”

Planteemos la idea de que un usuario es tan importante para como lo sean los productos que consume.

Vamos a caracterizar a cada usuario calculando el **PageRankMedio** de su cesta de la compra. Establecemos como usuario elite a aquellos situados en el top 10% superior de esta métrica.

```
# Nos quedamos con el producto y su pagerank
productos_pagerank <- grafo |>
  activate(nodes) |>
  as_tibble() |>
  select(name, PageRank)

# Mapeamos el pagerank al df original por usuario
usuarios <- df |>
  select(UserId, ProductId) |>
  inner_join(productos_pagerank, by = c("ProductId" = "name")) |>
  group_by(UserId) |>
  summarise(
    PageRankMedio = mean(PageRank),
    TotalResenas = n(),
    .groups = "drop"
  )

# Umbral de corte top 10%
elite <- quantile(usuarios$PageRankMedio, 0.90)

usuarios <- usuarios |>
  mutate(TipoUsuario = ifelse(
    PageRankMedio >= elite,
    "Usuario_Élite",
```

```

    "Usuario_Estándar"
  ))

resumen <- usuarios |>
  count(TipoUsuario, name = "CantidadUsuarios") |>
  mutate(Porcentaje = (CantidadUsuarios / sum(CantidadUsuarios)) * 100)

resumen

```

```

# A tibble: 2 x 3
  TipoUsuario      CantidadUsuarios Porcentaje
  <chr>              <int>         <dbl>
1 Usuario_Estándar    59953          89.9
2 Usuario_Élite       6765          10.1

```

Podemos ver que el mercado se divide en dos perfiles:

- **Usuarios Élite (top 10%):** Detectamos a los usuarios elite por la importancia de lo que compran. Son usuarios cuyos comentarios se concentran en los “Titanes” y “Líderes” de la red. Son los creadores de tendencias de Amazon: si ellos compran algo, es porque es un producto clave que mueve el mercado.
- **Usuarios Estándar (90% restante):** Es el consumidor base y común de la plataforma. Sus reseñas y compras se dispersan de forma totalmente aleatoria por productos periféricos o pequeños nichos minoritarios. Compran lo que necesitan en el día a día, por lo que su impacto individual a la hora de mover las tendencias de la red es casi nulo.

3.2 Interacción con IA

A continuación, se presenta un registro de los hitos más relevantes de la interacción con la IA, omitiendo las consultas redundantes y priorizando aquellos puntos donde la intervención humana fue crítica para el diseño del proyecto:

Para el desarrollo de este bloque, el uso de la IA ha sido especialmente intenso, convirtiéndose en un recurso clave sobre todo a la hora de estructurar y escribir el código en R. Al principio me encontraba perdido y no tenía una idea clara de lo que trataba el bloque y de cómo implementar los pipelines necesarios para conceptos como la homofilia.

3.2.1 Consulta 1

1. Prompt:

No entiendo muy bien que hay que hacer. Vamos paso a paso. Que hago primero y porque

En este caso le preguntaba sobre de que trataba el bloque ya que iba muy perdido. Tras la respuesta del prompt anterior, me preguntó si las categorías se asignan con su número o un nombre identificativo.

Mejor con el número de comunidad

2. Respuesta errónea o ineficiente de la IA:

Perfecto, mantener el número de comunidad original es la opción más limpia y robusta desde el punto de vista algorítmico, ya que evita errores de cadenas de texto y facilita la indexación posterior.

```
# =====
# Bloque 3: Clasificación Basada en Enlaces (Link-based)
# =====

# Paso 1: Crear las categorías artificiales en los nodos del grafo
grafo <- grafo |>
activate(nodes) |>
mutate(
  Categoria = if_else(
    Louvain %in% top_comunidades,
    paste0("Categoria_", Louvain),
    "Nichos_Minoritarios"
  )
)

# Verificación de control: Comprobamos el reparto total de los productos
tabla_control_categorias <- grafo |>
as_tibble() |>
count(Categoria, name = "productosAsignados") |>
arrange(desc(productosAsignados))

print(tabla_control_categorias)
```

3. Crítica:

La IA se saltó el “Protocolo de Fidelidad Estricta” que le impuse en el prompt inicial. Tenía prohibido escribir una sola línea de código a menos que yo usara la frase exacta “Necesito el código para esto” o se lo pidiera explícitamente, debiendo limitarse a usar el método socrático para hacerme pensar. Sin embargo, se olvidó de su faceta de tutor y me escupió el bloque de código entero directamente sin que nadie se lo pidiera.

4. Solución:

Aunque el código propuesto estaba bien para solucionar el problema, me sirvió para demostrar que la IA rompía constantemente sus propias normas, y tuve que responderle con un prompt para recordarle que no me diera ninguna línea de código a no ser que se lo pida.

3.2.2 Consulta 2

En esta ocasión, se destaca de forma constructiva cómo la IA fue capaz de realizar un desarrollo complejo (el pipeline de homofilia), resolviendo el problema con precisión y adaptándose por completo a las necesidades del proyecto.

1. Prompt:

uff explicamelo de otra forma porque no se como hacerlo en codigo

uff dame un poco de ayuda con código, dámelo

2. Respuesta eficiente de la IA:

¡Oído cocina! Alivio inmediato en camino. Tienes toda la razón, ver el pipeline estructurado en forma de “tablas cruzadas” es mil veces más fácil de programar que pelearse directamente con la memoria interna de un objeto de grafos.

Como has pedido explícitamente el código para desencallar este paso, aquí tienes la sintaxis exacta, limpia y comentada para tu documento Quarto. Este bloque utiliza exactamente tus variables (grafo) y la lógica de unir y agrupar dataframes que ya dominas.

```
# =====  
# Paso 3: Reto de Predicción por Homofilia Ponderada (Toda la Red)  
# =====  
  
# 1. Extraemos los nodos y aristas como tablas independientes de forma limpia  
df_nodos <- grafo |>  
  activate(nodes) |>  
  as_tibble() |>  
  mutate(id_nodo = row_number()) # Guardamos la posición original de cada producto
```

```

df_aristas <- grafo |>
  activate(edges) |>
  as_tibble()

# Como tu grafo NO es dirigido, un enlace conecta de 'from' a 'to' y viceversa.
# Duplicamos las filas invirtiendo los ejes para capturar TODOS los vecinos de cada producto
aristas_bidireccionales <- bind_rows(
  df_aristas |> select(nodo_origen = from, nodo_destino = to, weight),
  df_aristas |> select(nodo_origen = to, nodo_destino = from, weight)
)

# 2. PEGAR LAS CATEGORÍAS: Averiguamos la categoría artificial de cada nodo destino (vecino)
aristas_con_categoria <- aristas_bidireccionales |>
  inner_join(
    df_nodos |> select(id_nodo, Categoria_Vecino = Categoria),
    by = c("nodo_destino" = "id_nodo")
  )

# 3. LA VOTACIÓN: Agrupamos por producto origen y sumamos los pesos ('weight') de sus vecinos
votacion_ponderada <- aristas_con_categoria |>
  group_by(nodo_origen, Categoria_Vecino) |>
  summarise(Puntos_Totales = sum(weight), .groups = "drop")

# 4. EL GANADOR: Para cada producto origen, nos quedamos SOLO con la categoría más votada
predicciones_finales <- votacion_ponderada |>
  group_by(nodo_origen) |>
  # slice_max selecciona la fila con el máximo de puntos.
  # Si hubiera un empate absoluto en puntos, 'with_ties = FALSE' elige uno para no duplicar
  slice_max(order_by = Puntos_Totales, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(id_nodo = nodo_origen, Prediccion_Homofilia = Categoria_Vecino)

# 5. ACTUALIZAR EL GRAFO: Unimos las predicciones con nuestra lista de nodos original
df_nodos_actualizado <- df_nodos |>
  left_join(predicciones_finales, by = "id_nodo") |>
  # Si algún producto hubiera quedado aislado sin vecinos, le ponemos una etiqueta de categoría
  mutate(Prediccion_Homofilia = if_else(is.na(Prediccion_Homofilia), "Sin_Vecinos", Prediccion_Homofilia))
  arrange(id_nodo) # Nos aseguramos de mantener el orden exacto de las filas

# Inyectamos la nueva columna de predicción de vuelta en los nodos de tu grafo
grafo <- grafo |>
  activate(nodes) |>
  mutate(Prediccion_Homofilia = df_nodos_actualizado$Prediccion_Homofilia)

```

```
# =====
# AUDITORÍA DEL MODELO: Evaluación de Resultados
# =====

# Filtramos los productos que sí tienen vecinos para hacer un análisis justo
datos_auditoria <- grafo |>
  activate(nodes) |>
  as_tibble() |>
  filter(Prediccion_Homofilia != "Sin_Vecinos")

# Generamos la Matriz de Confusión (Tabla cruzada Realidad vs Predicción)
matriz_confusion <- table(
  Real = datos_auditoria$Categoria,
  Prediccion = datos_auditoria$Prediccion_Homofilia
)

# Calculamos el porcentaje global de acierto (Accuracy)
accuracy_global <- sum(diag(matriz_confusion)) / sum(matriz_confusion) * 100

# Mostramos los resultados en la consola de Quarto
print("--- MATRIZ DE CONFUSIÓN (HOMOFILIA PONDERADA) ---")
print(matriz_confusion)

cat("\n--- PORCENTAJE GLOBAL DE ACIERTO (ACCURACY) ---\n")
print(paste0(round(accuracy_global, 2), "%"))
```

3. Crítica:

Esta interacción representó el mayor acierto de asistencia de todo el bloque. El modelo no solo evitó alucinaciones, sino que demostró una comprensión impecable del grafo que estamos desarrollando. Además fue una de las pocas interacciones que me dejó claro todas las dudas que tenía sobre la homofilia.

4. Solución:

El pipeline propuesto se integró y compiló a la primera sin un solo aviso de error y devolvió la matriz de confusión que sirvió para justificar el 99.91% de Accuracy del modelo de homofilia.